Mockito Hands-On Exercises

Exercise 1: Mocking and Stubbing

Scenario:
You need to test a service that depends on an external API. Use Mockito to mock the external API and stub its methods.
Steps:
1. Create a mock object for the external API.
2. Stub the methods to return predefined values.
3. Write a test case that uses the mock object.
Solution Code:

```
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
public class MyServiceTest {
@Test
public void testExternalApi() {
ExternalApi mockApi = Mockito.mock(ExternalApi.class);
when(mockApi.getData()).thenReturn("Mock Data");
MyService service = new MyService(mockApi);
String result = service.fetchData();
assertEquals("Mock Data", result);
}
}
```

output:
ExternalApi.java

```
public interface ExternalApi {
    String getData();
}
```

MyService.java

```
public class MyService {
    private ExternalApi api;
    public MyService(ExternalApi api) {
        this.api = api;
    }
    public String fetchData() {
        return api.getData();
    }
}
```

MyServiceTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
```

```
public class MyServiceTest {
    @Test
    public void testExternalApi() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        when(mockApi.getData()).thenReturn("Mock Data");
        MyService service = new MyService(mockApi);
        String result = service.fetchData();
        assertEquals("Mock Data", result);
    }
}
```

Exercise 2: Verifying Interactions

Scenario:

You need to ensure that a method is called with specific arguments.

Steps:

1. Create a mock object.
2. Call the method with specific arguments.
3. Verify the interaction.

Solution Code:

```
import static org.mockito.Mockito.*; import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
public class MyServiceTest {
@Test
public void testVerifyInteraction() {
ExternalApi mockApi = Mockito.mock(ExternalApi.class);
MyService service = new MyService(mockApi);
service.fetchData();
verify(mockApi).getData();
}
}
```

Mock object:

```
ExternalApi mockApi = Mockito.mock(ExternalApi.class);
```

ExternalApi.java

```
public interface ExternalApi {
    String getData();
}
```

MyService.java

```
public class MyService {
    private ExternalApi api;
    public MyService(ExternalApi api) {
        this.api = api;
    }
    public void fetchData() {
        api.getData();
    }}
```

MyServiceTest.java

```
import static org.mockito.Mockito.verify;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
public class MyServiceTest {
    @Test
    public void testVerifyInteraction() {
        // Step 1: Create a mock object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        // Step 2: Inject the mock into the service
        MyService service = new MyService(mockApi);
        // Step 3: Call the method
        service.fetchData();
        // Step 4: Verify that getData() was called
        verify(mockApi).getData();
    }
}
```

Exercise 3: Argument Matching

Scenario:

You need to verify that a method is called with specific arguments.

Steps:

1. Create a mock object.
2. Call the method with specific arguments.
3. Use argument matchers to verify the interaction.

ExternalApi.java

```
public interface ExternalApi{
void sendData(String data);
}
```

MyService.java

```
public class MyService{
private ExternalApi api;
public MyService(ExternalApi api){
this.api=api;
}
public void processData(String data){
api.sendData(data);
}
}
```

MyServiceTest.java

```
import static org.mockito.Mockito.*;
import static org.mockito.ArgumentMatchers.*;
import org.junit.jupiter.api.Test;
public class MyServiceTest{
@Test
```

```
public void testArgumentMatching(){
ExternalApi mockApi=mock(ExternalApi.class);
MyService service=new MyService(mockApi);
service.processData("Hello");
verify(mockApi).sendData(eq("Hello"));
}
}
```

Exercise 4: Handling Void Methods

Scenario:
You need to test a void method that performs some action.
Steps:
1. Create a mock object.
2. Stub the void method.
3. Verify the interaction.

ExternalApi.java

```
public interface ExternalApi{
void deleteData();
}
```

MyService.java

```
public class MyService{
private ExternalApi api;
public MyService(ExternalApi api){
this.api=api;
}
public void removeData(){
api.deleteData();
}
}
```

MyServiceTest.java

```
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
public class MyServiceTest{
@Test
public void testVoidMethod(){
ExternalApi mockApi=mock(ExternalApi.class);
doNothing().when(mockApi).deleteData();
MyService service=new MyService(mockApi);
service.removeData();
verify(mockApi).deleteData();
}
}
```

Exercise 5: Mocking and Stubbing with Multiple Returns

Scenario:
You need to test a service that depends on an external API with multiple return values.

Steps:
1. Create a mock object for the external API. 2. Stub the methods to return different values on consecutive calls.
3. Write a test case that uses the mock object.

ExternalApi.java

```
 public interface ExternalApi{
String getData();
}
```

MyService.java

```
public class MyService{
private ExternalApi api;
public MyService(ExternalApi api){
this.api=api;
}
public String fetchData(){
return api.getData();
}
}
```

MyServiceTest.java

```
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
public class MyServiceTest{
@Test
public void testMultipleReturns(){
ExternalApi mockApi=mock(ExternalApi.class);
when(mockApi.getData()).thenReturn("First","Second","Third");
MyService service=new MyService(mockApi);
assertEquals("First",service.fetchData());
assertEquals("Second",service.fetchData());
assertEquals("Third",service.fetchData());
}
}
```

Exercise 6: Verifying Interaction Order

Scenario:
You need to ensure that methods are called in a specific order.
Steps:
1. Create a mock object.
2. Call the methods in a specific order.
3. Verify the interaction order.

ExternalApi.java

```
public interface ExternalApi{
void connect();
void disconnect();
```

```
}
MyService.java public class MyService{
private ExternalApi api;
public MyService(ExternalApi api){
this.api=api;
}
public void execute(){
api.connect();
api.disconnect();
}
}
MyServiceTest.java import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import org.mockito.InOrder;
public class MyServiceTest{
@Test
public void testInteractionOrder(){
ExternalApi mockApi=mock(ExternalApi.class);
MyService service=new MyService(mockApi);
service.execute();
InOrder inOrder=inOrder(mockApi);
inOrder.verify(mockApi).connect();
inOrder.verify(mockApi).disconnect();
}
}
```

Exercise 7: Handling Void Methods with Exceptions

Scenario:

You need to test a void method that throws an exception.

Steps:

1. Create a mock object.
2. Stub the void method to throw an exception.
3. Verify the interaction.

ExternalApi.java

```
public interface ExternalApi{
void deleteData();
}
MyService.java
public class MyService{
private ExternalApi api;
public MyService(ExternalApi api){
this.api=api;
}
public void removeData(){
```

```
api.deleteData();
}
}
```

MyServiceTest.java

```
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
public class MyServiceTest{
@Test
public void testVoidMethodException(){
ExternalApi mockApi=mock(ExternalApi.class);
doThrow(new RuntimeException("Error")).when(mockApi).deleteData();
MyService service=new MyService(mockApi);
assertThrows(RuntimeException.class,()->service.removeData());
verify(mockApi).deleteData();
}
}
```